TAFG v2: The Alchemist's Workbench — Project Blueprint

Document Information

- **Version:** 1.2
- **Date:** October 25, 2025

1. Project Mission & Philosophy

Mission

Create a single-page web application as a persistent, personal "workbench" based on "The Alchemist's Field Guide" (TAFG).

Philosophy

1.5 Global UI/UX & Robustness Requirements

Features required across all modules for usability:

- **Empty States:**
  Graceful handling for first-time users. All lists (Goals, Bricks) display a helpful message (e.g.,

  > "You have no Goals yet. Create your first one in the 'Comprehend' tab!")

- **CRUD Operations:**
  Users can Create, Read, Update (Edit), and Delete their own Goals and Bricks.

- **Soft Delete & Undo:**
  Deleting sets a flag (`isDeleted: true`). A "toast" notification appears for 5–10 seconds with an "Undo" button.

- **User Feedback:**
  Clear, non-technical feedback throughout.

- **Loading States:**
  Show a spinner when fetching or saving data.

- **Success Messages:**
  Brief confirmation (e.g.,

  > "Brick saved!")

- **Error Handling:**
  Friendly message if save fails (e.g.,

> "Could not save. Please check your connection.")

---

2. Technology Stack (The "How")

This project is a single HTML file using a Backend-as-a-Service (BaaS) for data and authentication.

Component	Technology	Choice & (Pros/Cons)
Frontend File	HTML	`TAFG_v2.html` (single file)
Styling	CSS	Tailwind CSS (CDN) — Rapid, modern UI; CDN load
App Logic	JavaScript	Vanilla JS (ES6+) — No build step, fast, all-in-one-file; code can get large
Charts	JS Library	Chart.js (CDN) — Excellent for "Energy Log" chart
Database	BaaS	Google Firestore — Real-time, generous free plan, auto-scaling; requires setup
Authentication	BaaS	Google Firebase Auth — Secure, easy "Sign in with Google," user management
Hosting	Static Host	Vercel or GitHub Pages — Free, fast, integrates with GitHub repo

---

3. Core Setup: Firebase (The Critical Path)

One-time setup steps:

1. **Create Project:**
   Firebase Console → New project (e.g., "Alchemist Workbench")
2. **Add Web App:**
   Dashboard → `</>` icon → Add "Web App"
3. **Get Config:**
   Copy `firebaseConfig` object for use in `TAFG_v2.html`
4. **Enable Authentication:**
   Build → Authentication → Sign-in method → Enable Google provider
5. **Enable Database:**
   Build → Firestore Database → Create database (start in "production mode")
6. **Set Security Rules:**
   Firestore → Rules tab → Paste the following (ESSENTIAL for privacy):

```
rules_version = '2';
service cloud.firestore {
    match /databases/{database}/documents {
        // Allow users to read/write *only* their own documents in each
collection
```

```
        match /goals/{docId} {
            allow read, write, delete: if request.auth != null &&
request.auth.uid == resource.data.userId;
        }
        match /bricks/{docId} {
            allow read, write, delete: if request.auth != null &&
request.auth.uid == resource.data.userId;
        }
        match /notes/{goalId} {
            // notes docId will BE the goalId
            allow read, write, delete: if request.auth != null &&
request.auth.uid == resource.data.userId;
        }
        match /energyLog/{docId} {
            allow read, write, delete: if request.auth != null &&
request.auth.uid == resource.data.userId;
        }
        // Add rules for new collections from v1.2 blueprint
        match /dailyLog/{docId} {
            allow read, write, delete: if request.auth != null &&
request.auth.uid == resource.data.userId;
        }

        // Allow users to *create* new documents, but only if they set the
userId to their own
        match /goals/{docId} {
            allow create: if request.auth != null &&
request.resource.data.userId == request.auth.uid;
        }
        match /bricks/{docId} {
            allow create: if request.auth != null &&
request.resource.data.userId == request.auth.uid;
        }
        match /notes/{goalId} {
            allow create: if request.auth != null &&
request.resource.data.userId == request.auth.uid;
        }
        match /energyLog/{docId} {
            allow create: if request.auth != null &&
request.resource.data.userId == request.auth.uid;
        }
        // Add create rule for new collections from v1.2 blueprint
        match /dailyLog/{docId} {
            allow create: if request.auth != null &&
request.resource.data.userId == request.auth.uid;
        }
    }
}
```

---

4. Application Modules (The "What")

The UI is a tabbed interface for a 3-step process:

Module 1: Authentication

- **UI:**
  - "Sign in with Google" button on loading screen
  - "Sign Out" button in header
- **Logic:**
  - Uses Firebase Auth
  - On login, queries Firestore for user data

Module 2: "Comprehend" (Your Goals)

- **UI:**
  - "Project List" tab
  - Empty state message if no goals
  - Text box:

    > "What's your 'One True Thing'?"

  - Button: `[ Alchemize (Create Goal) ]`
  - List of current goals (`isDeleted == false`) with `[Edit]` and `[Delete]` buttons
- **Logic:**
  - "Alchemize" creates a new goal
  - List populated from goals collection
  - Clicking a goal makes it "active"

Module 3: "Deconstruct" (Your Workbench)

- **UI:**
  - Main "work" tab showing "Active Goal"
  - **Part 1: "Your One Brick"**
    - Text box:

      > "What's the absolute next physical step?"

    - Button: `[ Add Brick ]`
    - List of incomplete Bricks (`isDeleted == false`)
    - Each brick: `[ ✓ ]` checkbox, `[Edit]`, `[Delete]`
  - **Part 2: "Brain Dump" (The Note Pad)**
    - Large text area below "One Brick"
- **Logic:**
  - Adding a Brick creates a new document
  - Checking `[ ✓ ]` updates `isComplete`
  - "Brain Dump" auto-saves to notes

Module 4: "Reconstruct" (Your Log)

- **UI:**
  - "Motivation" tab
  - Empty state if no completed bricks
  - Reverse-chronological list of completed Bricks (`isComplete == true`, `isDeleted == false`)
- **Logic:**
  - Read-only query of bricks collection
  - Shows tangible proof of progress

Module 5: "Two-Front War" (Your Energy)

- **UI:**
  - "Energy" tab
  - Slider (Healing % vs. Building %)
  - Button: `[ Log Today's Energy ]`
  - Empty state if no logs
  - Chart.js line chart of energy log history
- **Logic:**
  - Button creates new energyLog document
  - Chart reads from energyLog collection

---

5. Data Structure (Firestore Collections)

How data is organized:

- **goals/**
  `{goalId_A}: { userId: "...", title: "Start YouTube Channel", createdAt: ..., isDeleted: false }`
- **bricks/**
  `{brickId_A}: { userId: "...", goalId: "goalId_A", task: "Open phone camera", isComplete: false, createdAt: ..., isDeleted: false }`
  `{brickId_B}: { userId: "...", goalId: "goalId_A", task: "Google 'free video software'", isComplete: true, completedAt: ..., isDeleted: false }`
- **notes/**
  (Use goalId as document ID)
  `{goalId_A}: { userId: "...", content: "auto-saved brain dump text...", updatedAt: ... }`
- **energyLog/**
  `{logId_A}: { userId: "...", date: "2025-10-25", healing: 70, building: 30 }`
- **dailyLog/**
  (For future v3.5)
  `{logId_A}: { userId: "...", date: "2025-10-25", mood: 4, note: "Good day, felt productive." }`

---

6. Future Roadmap (The "v3" Plan)

Features postponed to ensure v2 is built correctly ("One-Brick" rule):

- **v3.0: Project Sharing ("Friends List")**
  - Share a goalId with another user by email
  - Requires new shares collection and complex security rules
- **v3.1: Advanced Statistics**
  - "Stats" tab: Bricks completed per week, most productive day, etc.
- **v3.2: PWA (Progressive Web App)**
  - Service worker and manifest for installable app-like experience
- **v3.3: Framework Migration (e.g., Next.js)**
  - Move from single-file to full-stack framework for scalability and advanced features
- **v3.4: "Trash Can" (Recovery)**
  - UI for viewing items with `isDeleted: true`
  - Permanently delete or restore items
- **v3.5: "Daily Log / Mood Journal"**
  - Simple journal for daily mood score (1–5) and note
  - Complements energyLog for qualitative self-reflection